Contents

Qminers 2026: My Practical Field Guide **1**
0. What I actually know before the event . . . 1
1. What professional trading teams do — in normal words . . . 2
2. The fast problem classifier . . . 2
3. Module 01 — Market understanding: terms I actually need . . . 3
4. Module 02 — Machine learning without guessing . . . 4
5. Module 03 — Game theory and resource allocation . . . 5
6. Module 04 — Coding and automation . . . 6
7. How our research workflow should work . . . 7
8. Team operations in a 24-hour contest . . . 8
9. How to use Codex wisely . . . 8
10. What is worth reading — the shortest useful set . . . 9
11. My offline emergency checklist . . . 9
12. Twenty questions I should be able to answer . . . 9
Sources and why they matter . . . 10

Qminers 2026: My Practical Field Guide

Prepared 3 November 2026 | Competition 7–8 November | Personal study copy

> I do not need to memorize a hundred models. I need to recognize the problem, start with something correct, test it, and improve it without losing the working solution.

0. What I actually know before the event

The Qminers 2026 public description says we will build an algorithm to trade in a simulated market. My own planning README lists four themes: Market Understanding, Machine Learning, Game Theory & Strategy, and Coding & Automation. Those names are NOT proof that four separate problems will appear. The full official rules/API were not readable through the public browser when this guide was prepared. **The first job on competition day is to read the actual task and rules.**

I am preparing broadly because past student competitions have included: market-making simulations (Optiver Ready Trader Go), sports betting / prediction plus risk decisions (Qminers 2024–25), simulated trading plus manual math games (IMC Prosperity), auctions, resource allocation, and exchange/arbitrage puzzles. They are examples, not forecasts of the Qminers task.

Five questions before writing any serious code

1. What exactly is the input? Files, arrays, a stream of market events, game states, or a network API?
2. What is the output? A prediction, a price, a set of orders, a resource allocation, a single number, or one Python file?
3. How do we score? Profit, Sharpe ratio, rank, expected reward, probability of winning, runtime, or a mixture?
4. What can we use? Provided data only, internet, prewritten code, libraries, memory between calls, multiple threads?
5. How can we test? Official evaluator, sample cases, replay files, hidden leaderboard, or only local reasoning?

If any of these is unclear, we write it down as UNKNOWN, not as a fact.

1. What professional trading teams do — in normal words

This is based on public descriptions by Optiver and Jane Street, not inside information about their secret strategies.

They estimate value. A price on the screen is not automatically a good price. A trading model tries to work out what something is worth, given the available information.

They measure what can go wrong. A strategy may make small profits a hundred times and then lose everything in one bad market move. They track positions, order exposure and losses before they happen.

They separate ideas from execution. A prediction is not a trade. A profitable-looking trade can be ruined by transaction costs, delays or an order that never gets filled.

They record and review experiments. If changing a parameter improves a backtest, we must know the old and new settings, data period and exact test. A single good result is not proof.

They keep working systems working. We need a boring, valid backup solution while testing a more ambitious one. Every change must have a way back.

For this hackathon we copy the discipline, not the scale. We do not need nanosecond hardware or a massive neural network to use these principles.

2. The fast problem classifier

I see…	The problem is probably…	I start with…	I must check…
Bid/ask prices, order IDs, fills, cancels	Order-book trading	Safe fixed quotes + position limits	How orders fill; penalties and fees
A single noisy time series	Prediction or trading signal	Last value, mean, EWMA, linear model	Future leakage and trading costs
Tables with outcomes to predict	Supervised ML	Majority/mean baseline, logistic/ridge	Train/test split and metric
Prices of two linked assets	Relative-value / pair trading	Difference or ratio, fees-aware threshold	Are both assets actually tradable?
Basket price and component prices	Basket arbitrage	Sum the component values	All legs, fees, volume limits
Budget, items, costs, rewards	Knapsack / allocation	Brute force small, DP or MILP large	Integer or fractional? Constraints?
Many players choose a limited reward	Congestion / game theory	Expected reward under competitor models	Reward-sharing rule and uncertainty
Other players submit secret bids	Auction	Grid search expected payoff	Auction type, payment rule, ties
Payoff matrix	Matrix game	Best response and LP if zero-sum	Is it actually zero-sum?
States, actions, probabilities	MDP / sequential decision	Bellman recursion or rollout	Is the horizon finite?
Currency exchange rates	Graph optimization	Convert-path enumeration / negative logs	Fees, limits, number of conversions
Strict method signature, memory/runtime	Programming task	Exact reference implementation	Packaging, allowed imports and edge cases

The classifier is a suggestion, not an answer. When in doubt, we first implement the obvious baseline and use the provided evaluator.

3. Module 01 — Market understanding: terms I actually need

Order book

An order book shows prices at which someone currently wants to buy (**bid**) and sell (**ask**). Example:

```
ASK: 101.0   quantity 8
ASK: 100.5   quantity 15   <- best ask
BID: 100.0   quantity 12   <- best bid
BID:  99.5   quantity 20
```

The spread is 100.5 - 100.0 = **0.5**. The midprice is (100.5 + 100.0)/2 = **100.25**. Buying immediately usually means paying the ask, not the midprice. Selling immediately usually means taking the bid.

A **market order** asks to trade now and accepts the available price; it can suffer slippage. A **limit order** specifies a worst acceptable price, but may not execute. A submitted order is not the same as a filled order.

Inventory means my net position. If I bought 10 and sold 7, my inventory is +3. When I already own too much, I should usually be less eager to buy even more and more eager to sell. This is inventory-aware quoting.

Adverse selection means the other side of my trade knows something, or reacts faster. Example: my buy quote gets filled repeatedly just before the market drops. A fill is not necessarily good news.

Microprice weights the two best prices by opposite-side quantities. If bid volume is 30 and ask volume is 10, a common estimate is:

`microprice = (ask_price * bid_volume + bid_price * ask_volume) / (bid_volume + ask_volume)`.

Here it is (100.5 × 30 + 100 × 10)/40 = **100.375**, slightly above the midprice. It is a rough signal, not a guarantee. A volume imbalance at the top of the book can disappear quickly.

Order-flow imbalance measures recent pressure from changes in bid/ask quotes or trades, depending on the exact definition. It requires the appropriate event data. We must not calculate pretend order-flow events from sparse book snapshots.

Fair value means our best estimate of price, not the observed midprice by definition. We can start with an EWMA or a simple signal model. If I think fair value is 100.2 and the best ask is 100.4, immediately buying is not obviously profitable: I would pay more than my estimate.

Scenario M1 — The market stays near one price

What I see: price keeps coming back toward a fairly stable level.

What it might mean: mean reversion. Market makers may earn the spread if execution and risk are reasonable.

What I do: plot a rolling mean; estimate a deviation score; test a simple buy-low/sell-high policy or quotes around fair value; add position caps.

How I check: time-split data; include actual fill assumptions and costs; compare against doing nothing and a fixed-spread market maker.

When I stop: returns are caused by unrealistic fill assumptions, or the price drifts away and inventory grows dangerously.

Scenario M2 — The market trends strongly

What I see: short-horizon returns keep the same sign over several windows.

What it might mean: momentum or a regime change. The strategy that sells every rise may suffer.

What I do: start with a sign-of-recent-return baseline, use a volatility filter and position cap; compare against mean reversion on future periods.

How I check: performance by regime and after fees; avoid fitting the trend window on the final test period.

Scenario M3 — We must quote into a live book

What I see: callbacks for book updates, order acknowledgements, fills and cancellations.

What I do: first run a tiny, safe baseline; log every order state; verify cash/position accounting. Only then add microprice or order-flow signals. Quoting parameters are NOT the first problem; a broken fill handler is.

How I check: deliberately test partial fills, late cancel acknowledgements, stale messages, double callbacks and near-position-limit orders.

Scenario M4 — Two assets move together

What I see: an ETF/future, two correlated commodities, or a basket and its parts.

What I do: estimate a fair spread or fair basket price; act only if an executable price exceeds estimated value **after** all costs. I must count the risk if only one side fills.

How I check: out-of-sample spread stability, position limits per asset and worst-case one-sided fill.

Scenario M5 — Our strategy makes many fills but loses money

Likely reasons: adverse selection, hidden costs, overly optimistic fair value, poor accounting, trend, slow updates.

What I do: split PnL into spreads earned, mark-to-market price movement and fees; measure the midprice 1/5/20 steps after each fill; compare fill quality against no-trade outcomes.

Do not respond by blindly increasing order size.

4. Module 02 — Machine learning without guessing

I do not need a huge model first. I need a clean comparison.

Baseline: a trivial rule. For predicting tomorrow's price, predict today's price. For predicting a binary event, start with the overall frequency or a simple logistic regression.

Features: the information the model sees, for example spread, volatility, imbalance, previous outcomes and volume. Every feature must have been available before the moment of prediction.

Target: what I want to predict, for example future midprice movement or probability that a team wins. The target's time must be in the future of the features.

Leakage: an accidental shortcut, such as using the game's final score to predict who wins or a price update from after the trade. Leakage makes a backtest look amazing and breaks in practice.

Training / validation / test: training fits the model, validation chooses settings, test gives a final unbiased check. For time series the order must be chronological. Randomly shuffling adjacent market observations often makes validation misleading.

Calibration: if I predict a probability of 70% many times, approximately 70% of those cases should occur. Probability accuracy is important when sizing bets or trading decisions.

Scenario L1 — A CSV of observations and answers

1. Inspect rows, columns, missing values, timestamps and target frequency.
2. Reproduce the exact scoring metric. Do not assume accuracy if scoring is log loss, PnL or Brier score.
3. Split data in time or by independent groups as appropriate.
4. Run a constant baseline, then logistic/ridge, then one tree model.
5. Plot errors and analyze the worst cases. Add features that have a reason to work.
6. Test a small parameter search. Keep a final holdout untouched.
7. Package only the dependencies allowed by the submission environment.

Scenario L2 — A predictive signal with tiny edge

Predicting up/down 51% correctly may be useful or completely useless. If costs are 2 ticks and the average gain is 0.5 tick, the strategy loses money. We must convert predictions into **net executable trading decisions** and simulate inventory/risk.

Scenario L3 — Too little data / a constantly changing market

Prefer small regularized models and simple features. Large neural networks are hard to justify without enough independent examples. Compare performance over multiple time periods rather than trusting an aggregate metric.

5. Module 03 — Game theory and resource allocation

Scenario G1 — I have a budget and must choose items

Example: I have 10 coins. Project A costs 6 and pays 11, B costs 4 and pays 8, C costs 5 and pays 9. Under a one-time choice constraint, A+B costs 10 and pays **19**, which is better than A alone, B+C (17), or C alone. For a small problem, enumerate every valid combination; this becomes a reliable test for the DP or integer programming solver.

What I run: brute force for small n; knapsack DP or mixed-integer linear programming for larger n.

What I check: all constraints, integer vs fractional units, fees, whether unused budget has value.

Scenario G2 — Other players share my reward

Example: Reward A = 100 and B = 80. If I expect nine other players to choose A and one other player to choose B, my estimated share is A: 100/10 = **10**, B: 80/2 = **40**. B looks better despite its smaller headline reward.

Problem: those are guesses about other people. A different distribution may reverse the choice.

What I run: compute expected payoff for each choice under several rival-behavior scenarios; simulate uncertain crowd sizes; consider a mixed strategy if the game rewards being unpredictable.

What I check: exact sharing formula, whether extra picks cost money, whether players can see earlier choices and respond.

Scenario G3 — We must submit a bid

First-price auction: winner pays their own bid. Second-price auction: winner typically pays the second-highest bid. All-pay auction: everybody pays their bid even if they lose. **These are very different optimization problems.**

What I do: write expected profit as a function of my bid, simulate other bids using plausible distributions, compare the best bid across those distributions, inspect tie-breaking rules. Do not use a formula from a different auction type.

Scenario G4 — The statement gives a payoff matrix

I write the rewards for every combination of actions. If it is a two-player zero-sum game, I can compute a mixed-strategy equilibrium with a linear program. If not zero-sum, I may need best responses or a different equilibrium method.

Why mixed strategies matter: always playing my 'best' action can make me predictable and exploitable.

What I check: minimize my opponent's best possible advantage, not merely maximize my score against one guessed opponent.

Scenario G5 — Multiple rounds and a leaderboard

The objective might be **probability of finishing first**, not expected profit. A risky play may be worthwhile when far behind, but unnecessary when comfortably leading. This was central to the Qminers 2024 winner's published tournament-risk simulation.

What I do: simulate many plausible final rounds; compare safe/moderate/aggressive action plans; report expected score, probability of losing everything and probability of first place. Record assumptions about opponents.

Scenario G6 — A sequence of decisions changes future options

If choosing now changes the next state, use backward induction or dynamic programming for small finite-horizon problems. If the branching is huge, use bounded Monte Carlo rollouts. Always test a toy problem where you can enumerate every path.

Scenario G7 — Exchange rates or conversion opportunities

If converting A→B→C→A yields more A than I started with after fees, there may be an arbitrage cycle. Taking `-log(rate)` converts products of rates into sums, making shortest-path/negative-cycle techniques possible. But if conversions are limited to a few steps, a simple dynamic program or bounded search may be clearer.

6. Module 04 — Coding and automation

The submission contract always comes first. A perfect algorithm with the wrong class name or invalid import is worth zero.

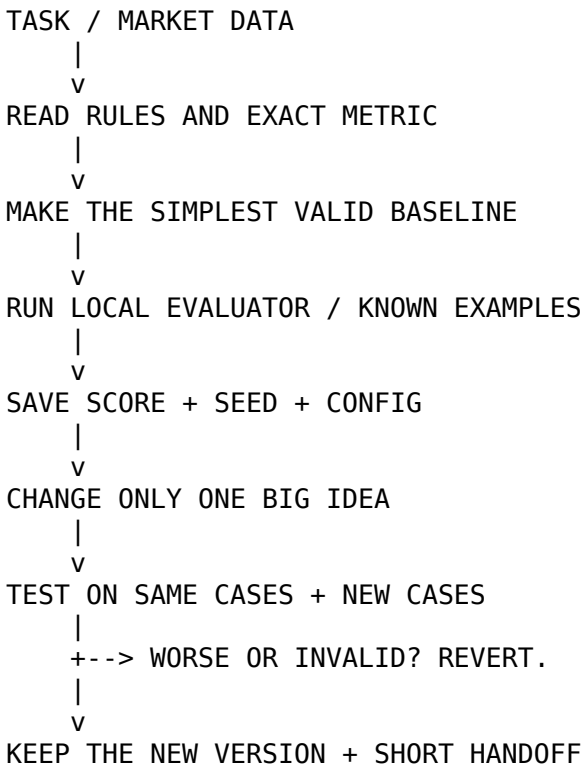
Before optimizing: run the official sample, confirm runtime, memory, filenames, function signatures, Python version, stdout/stderr restrictions and allowed external packages.

Quick algorithm map:

Shape of problem	First tool
Choose best valid subset	Enumeration, knapsack DP, MILP
Find minimum-cost routes	Dijkstra (nonnegative edges), Bellman-Ford (negative edges)
Reachability / connected components	BFS/DFS/union-find
Repeated sums on ranges	Prefix sums
Search for a threshold	Binary search
Many repeated states	Memoization / DP
Continuous constrained allocation	SciPy optimize / LP / QP
Small random system	Exact enumeration first; Monte Carlo second
Large repeated numerical operations	NumPy vectorization; profile before JIT

Important: algorithm correctness and runtime are different problems. I first get the right answer on small tests, then compare speed on realistic input sizes.

7. How our research workflow should work



We do not choose a strategy because it is fashionable or because the code looks sophisticated. We choose it because it repeatedly performs better on the actual scoring rule and passes the allowed validation.

Reading a result in plain English

- **Profit**: how much money the simulation says we made. Inspect the accounting.
- **Drawdown**: how far the portfolio fell from its previous peak. A large drawdown may kill us before profits arrive.
- **Fill rate**: fraction of submitted orders that actually traded; high isn't always good.
- **Sharpe ratio**: return compared with its volatility. A high estimate can be meaningless with little or dependent data.
- **Log loss / Brier**: how good predicted probabilities are, not just correct/incorrect labels.
- **Confidence interval**: range reflecting sampling uncertainty, not a promise that the future sits inside it.
- **Exploitability**: how much a best-responding opponent could gain because our strategy is predictable.

What I want on each PDF report

A headline stating the actual question; dataset/date and sample size; a graph with real units; baseline vs proposed solution; costs and constraints; repeatability details; known reasons it may fail; a one-sentence practical next step. **Example data must be labeled as example data.**

8. Team operations in a 24-hour contest

Before coding

One person owns the rules and exact scoring. One person owns the submission interface and working baseline. One person owns experiments/analysis. A fourth person (if available) owns game theory/optimization or independent QA. With fewer people, combine responsibilities. These are roles, not rigid titles.

First 30 minutes

- Everybody reads the task and checks understanding together.
- Agree on the evaluation metric and legal constraints.
- Create a working, valid submission early.
- Assign modules and files so people do not overwrite each other.
- Put a simple benchmark and command in the README.

During development

- Short sync every 60–90 minutes: what improved, what failed, what's next.
- One experiment question per branch. Prefer short commits.
- One person merges and keeps the last-known-good version.
- Nobody rewrites the submission interface without coordinating.
- Keep a compact `current_state.md`: best score, exact commit, open risks, next step.

Last 2–3 hours

Stop speculative rewrites. Re-run the complete submission from a clean environment. Check official restrictions. Submit with time for retries. Preserve the previous known-good version. Check which version the system actually accepted.

If I'm the least experienced person in the team

I can still contribute a lot: make a reproducible evaluator, run ablations, test corner cases, turn assumptions into tiny examples, write the integration checklist and spot changes that only worked on one seed. I should ask 'What evidence shows this helps?' rather than pretending I understand every advanced model.

9. How to use Codex wisely

Useful requests:

- 'Read the official specification, list verified rules and unknowns, then make the smallest valid submission.'
- 'Explain this trading signal with one numerical example. What would make it fail?'
- 'Run the old and new strategy on identical seeds and produce a one-page comparison.'
- 'Do not change strategy code. Audit the PnL ledger and fill lifecycle.'
- 'Here is an auction. Write the exact payoff formula before any implementation.'
- 'Build an exhaustive tiny-case oracle and test the optimized solver against it.'
- 'Generate a PDF from experiment 42; cite actual files and mark synthetic inputs.'

Less useful requests: 'Make me a genius bot', 'use deep learning everywhere', 'optimize everything', 'download every paper and build all strategies'. A narrow measured goal saves more usage than a giant confused run.

10. What is worth reading — the shortest useful set

One trading simulation first: Optiver Ready Trader Go. Run a sample bot, inspect match logs, see a fill and a cancellation. This teaches more about execution than reading many equations out of context.

One strategy write-up: IMC Prosperity 2024. Focus on fair value, the difference between maker and taker, inventory risk, and why a strategy that rarely fills can be worthless.

One game theory notebook: IMC Prosperity 2025 manual puzzle round 2 (reward sharing), or Prosperity 2024 manual write-ups. Work through a tiny example before reading the optimized approach.

One ML case: Qminers 2025 starter. Understand `Model.place_bets`, exact evaluation, a time-ordered train/test split and why restricted data matters.

Optional advanced theory: Avellaneda–Stoikov (inventory-sensitive quoting), Cont/Kukanov/Stoikov (order-flow imbalance), Stoikov's microprice work. Read formulas only after a baseline is running.

11. My offline emergency checklist

1. Can I state the objective and constraints in two sentences?
2. Did we confirm which rules apply to code and data?
3. Do we have a valid baseline submission right now?
4. Can I run the baseline from a fresh checkout?
5. Do I know the exact score and how it is computed?
6. Do we use only information available at decision time?
7. Does the model account for transaction costs/other players if relevant?
8. Is the latest improvement repeatable on independent cases?
9. Do we have a known-good commit and a way to revert?
10. Can the whole team explain what we are submitting?

12. Twenty questions I should be able to answer

1. **What's the spread?** Best ask minus best bid.
2. **What's midprice?** Average of best bid and best ask; not a guaranteed fill price.
3. **Why use limit orders?** To control price; downside is not getting filled.
4. **Why do market makers hold inventory?** Fills can be unbalanced; that position carries price risk.
5. **Why can a fill be bad?** It may occur just before price moves against us.
6. **What is fair value?** Our estimate of what the asset is worth, conditional on information.
7. **What is order imbalance?** Relative buying vs selling depth; a noisy possible signal.
8. **What is mark-to-market PnL?** Value current inventory at a reference market price plus cash, subject to the chosen convention.
9. **Why can a backtest lie?** Leakage, wrong fills, missing costs, selection bias, incorrect scoring.
10. **What is a baseline?** The simplest valid method we compare improvements against.
11. **Why not random split a time series?** Future observations can influence the past via correlations or leakage.
12. **What is overfitting?** Learning quirks of the development sample that do not persist.
13. **What's calibration?** Do forecast probabilities match observed frequencies?
14. **What's a constrained optimization problem?** Find the best decision while obeying limits.
15. **Why brute force sometimes wins?** It gives exact answers on small inputs and tests optimized methods.
16. **What is a mixed strategy?** Randomize among actions with chosen probabilities.
17. **What is a best response?** Best action against a particular opponent strategy.
18. **Why simulate rivals?** In shared-reward games, my payoff depends on what others do.
19. **Why save experiment configs?** So improvements can be checked and reproduced.
20. **What matters most near the deadline?** A valid, tested submission with a known score and rollback.

Sources and why they matter

- Qminers event description: https://qminers.com/en/hiring/competitions-events (2026 framing, not full rules).
- My planning notes: https://github.com/dmytriikorolov/qminers (four themes and links, user-maintained notes).
- Qminers 2024 winner's method: https://github.com/AdamBobocky/qminers-24 (prediction ensemble and tournament risk; betting, not 2026 market API).
- Qminers 2025 starter: https://github.com/dmytriikorolov/QH2025 (exact interface, evaluator and restrictions, not guaranteed to transfer).
- Optiver official how-to: https://readytradergo.optiver.com/how-to-play/ (simulator, exchange, risk and logs).
- Optiver source archive: https://github.com/trietvuive/optiver-ready-trader-go (event handlers and execution/risk limits; AGPL license).
- IMC 2024 strategy write-up: https://github.com/pe049395/IMC-Prosperity-2024 (fair value, maker/taker, inventory, arbitrage).
- IMC 2024 manual puzzle solutions: https://github.com/gabsens/IMC-Prosperity-2-Manual (probability and optimization).
- IMC 2025 manual solutions: https://github.com/nathanbronson/Prosperity2025 (reward sharing and optimization examples).
- IMC 2026 participant report: https://github.com/marcobattiston21/imc-prosperity-4 (auctions, resource allocation, options and trading ideas).
- Jane Street overview: https://www.janestreet.com/what-we-do/overview/ (technology, real-time risk visibility and teamwork).
- Optiver overview: https://www.optiver.com/what-we-do/ (pricing, execution and risk engineering).
- FIA automated-trading guide: https://www.fia.org/sites/default/files/2019-09/FIA-Guide-to-the-Development-and-Operation-of-Automated-Trading-Systems_1.pdf (risk controls, open orders and stale market data).

End of field guide. I can read any one scenario in two minutes and go from there.